

Course: Hoare Logic — Deductive Reasoning for Program Verification

Generated from user's notes

Table of contents

1	Introduction	2
2	1. Programs, specifications and correctness	2
3	2. Memory, expressions and operational semantics	2
4	3. Axiomatic semantics: Hoare logic and rules	3
4.1	3.1 Assignment rule (axiom schema)	3
4.2	3.2 Rule of consequence	3
4.3	3.3 Sequencing rule (composition)	3
4.4	3.4 Conditional rule	4
4.5	3.5 While rule (loop invariant)	4
4.6	3.6 Function call rule (contracts)	5
5	4. Weakest Precondition (WP)	5
6	5. The fundamental theorem: equivalence of Hoare validity and WP	6
7	6. Loops: invariants and variants (termination)	7
8	7. Functions and recursion	7
9	8. Worked examples	8
9.1	8.1 Addition by repeated increment (from notes)	8
9.2	8.2 Recursive sum (from notes)	8
10	9. Generating verification conditions (VCs)	9
11	10. Summary and practical advice	9

12 Appendix A — Full proof of substitution lemma (sketch)	10
13 Closing	10

1 Introduction

This course is a clean, self-contained explanation of **Hoare logic** and related deductive techniques for reasoning about programs. It is based on the notes you provided (where you used $\mathbf{m}$ to denote the program memory state instead of the usual μ), expanded with clarifications, proofs, and worked examples. The file is a Quarto `.qmd` document configured to render to PDF using the LuTeX engine.

We assume a small imperative language with integers and booleans, assignments, sequencing, conditionals, loops, and function calls. No explicit memory management is considered (no heaps, pointers, or allocation).

2 1. Programs, specifications and correctness

A *program* is a finite sequence of instructions written in our small language. A *specification* describes what the program is supposed to compute; we focus on **functional properties** (what the program computes), expressed as formulas of first-order logic (FOL).

A common way to specify a program fragment c is a **Hoare triple**

$$\{P\} c \{Q\}$$

where P (precondition) and Q (postcondition) are FOL predicates over program variables.

Meaning. The triple $\{P\} c \{Q\}$ is *valid* iff for every memory state m , if $m \models P$ and the execution of c from m terminates producing state m' , then $m' \models Q$. This is the usual notion of *partial correctness* (it does not assert termination).

3 2. Memory, expressions and operational semantics

We model the program memory as a function:

$$m : Var \longrightarrow \mathbb{Z} \cup \{\text{tt}, \text{ff}\}.$$

That is, each variable maps to an integer or a boolean value (true/false).

We define two key semantic functions:

- $Val(e, m)$: evaluates expression e in memory state m , returning a value in $\mathbb{Z} \cup \{\text{tt}, \text{ff}\}$.
- $Mem(c, m)$: the result state after executing instruction c from initial state m . If the execution diverges (non-terminating), $Mem(c, m)$ is undefined.

Basic update notation: $m[x \leftarrow v]$ denotes the memory identical to m except that x maps to v .

Example: $Mem(x := t, m) = m[x \leftarrow Val(t, m)]$.

4 3. Axiomatic semantics: Hoare logic and rules

Hoare logic gives **axioms** and **inference rules** to derive valid triples.

4.1 3.1 Assignment rule (axiom schema)

For an assignment $x := t$ the fundamental axiom is:

$$\{ Q[x \leftarrow t] \} x := t \{ Q \}$$

This reads: to ensure postcondition Q after assigning t to x , the precondition should be Q with every free occurrence of x replaced by expression t .

4.2 3.2 Rule of consequence

If we have $P' \Rightarrow P$ and $\{P\} c \{Q\}$ and $Q \Rightarrow Q'$, then we can derive:

$$\{P'\} c \{Q'\}$$

This lets us *strengthen* preconditions and *weaken* postconditions.

4.3 3.3 Sequencing rule (composition)

If $\{P\} c_1 \{R\}$ and $\{R\} c_2 \{Q\}$ then:

$$\{P\} c_1; c_2 \{Q\}$$

4.4 3.4 Conditional rule

For an if-then-else:

if b then c_1 else c_2

the rule is:

If $\{P \wedge b\} c_1 \{Q\}$ and $\{P \wedge \neg b\} c_2 \{Q\}$ then

$\{P\}$ if b then c_1 else $c_2 \{Q\}$

(Here b is a Boolean expression; $P \wedge b$ means P holds and the condition evaluates to true.)

4.5 3.5 While rule (loop invariant)

For the loop

while b do c done

the rule requires a loop invariant I :

If $\{I \wedge b\} c \{I\}$ then

$\{I\}$ while b do c done $\{I \wedge \neg b\}$

This establishes *partial correctness* relative to the invariant I . The invariant I must hold before the loop and be preserved by the body when the guard holds. On loop exit ($\neg b$), I together with $\neg b$ should imply the desired postcondition; this is checked separately (using the rule of consequence).

4.6 3.6 Function call rule (contracts)

A function f with contract $\{\phi\} f \{\psi\}$ means: whenever the parameters satisfy ϕ , an execution of f (if it terminates) yields result **res** satisfying ψ .

For a call $v := f(a)$, a sound rule is:

To ensure postcondition Q after $v := f(a)$, a sufficient precondition is:

$$\phi[a/\text{params}] \wedge \forall r. (\psi[a/\text{params}, \text{res} \leftarrow r] \Rightarrow Q[v \leftarrow r])$$

A common practical approach is to treat the call by introducing a fresh logical variable w to stand for the result and require:

$$\phi[a/\text{params}] \wedge \psi[a/\text{params}, \text{res} \leftarrow w] \wedge Q[v \leftarrow w]$$

and existentially quantify w if needed. In verification condition generation we typically inline contracts to produce checks handed to an SMT solver.

5 4. Weakest Precondition (WP)

The *weakest precondition* $WP(c, Q)$ is the weakest predicate P such that $\{P\} c \{Q\}$ holds. Formally:

$$WP(c, Q) \equiv \lambda m. \text{ if } Mem(c, m) \text{ is defined and equals } m' \text{ then } m' \models Q, \text{ else false.}$$

In practice we compute WP syntactically using the following rules (for deterministic terminating fragments):

- $$WP(x := t, Q) = Q[x \leftarrow t].$$
- $$WP(c_1; c_2, Q) = WP(c_1, WP(c_2, Q)).$$
- $$WP(\text{if } b \text{ then } c_1 \text{ else } c_2, Q) = (b \Rightarrow WP(c_1, Q)) \wedge (\neg b \Rightarrow WP(c_2, Q)).$$
- For loops, $WP(\text{while } b \text{ do } c, Q)$ is not (in general) computable. If an invariant I is provided and satisfies:
 - $\{I \wedge b\} c \{I\}$ (invariance)
 - $(I \wedge \neg b) \Rightarrow Q$ (postcondition on exit) then one can take $WP(\text{while } b \text{ do } c, Q) = I$.

Caveat. Precise computation of WP for loops (and recursion) would solve undecidable problems (e.g., termination/halting). Thus user-supplied invariants and variants are necessary for proofs.

6 5. The fundamental theorem: equivalence of Hoare validity and WP

Theorem. For any loop-free code c and FOL formulas P, Q :

$$\{P\} c \{Q\} \text{ is valid} \iff P \Rightarrow WP(c, Q)$$

This means: operational semantics and weakest-precondition semantics agree on partial correctness for loop-free programs.

Proof sketch (structural induction on c).

We work with memory states denoted m (following your notation).

1. **Base: assignment $x := t$.**

$WP(x := t, Q) = Q[x \leftarrow t]$ by definition.

- $(\Rightarrow)$ Suppose $\{P\} x := t \{Q\}$ is valid. Take any m with $m \models P$. By validity, $Mem(x := t, m) = m[x \leftarrow Val(t, m)] \models Q$. By substitution lemma (see below), this is equivalent to $m \models Q[x \leftarrow t]$, i.e. $m \models WP(x := t, Q)$. Thus $P \Rightarrow WP(x := t, Q)$.
- $(\Leftarrow)$ Suppose $P \Rightarrow Q[x \leftarrow t]$. For any m with $m \models P$, we have $m \models Q[x \leftarrow t]$, and by the substitution lemma $m[x \leftarrow Val(t, m)] \models Q$. But $m[x \leftarrow Val(t, m)] = Mem(x := t, m)$, so the triple is valid.

The key technical lemma used twice is:

Substitution lemma. For any expression a , variable x , expression t , and memory m ,

$$Val(a[x \leftarrow t], m) = Val(a, m[x \leftarrow Val(t, m)]).$$

And for any FOL formula Q ,

$$m \models Q[x \leftarrow t] \iff m[x \leftarrow Val(t, m)] \models Q.$$

This lemma is proved by structural induction on expressions and on formula structure.

2. **Inductive step: sequence and conditional.**

- Sequence $c_1; c_2$: by induction hypothesis $P \Rightarrow WP(c_1, WP(c_2, Q))$ is equivalent to validity of $\{P\} c_1; c_2 \{Q\}$.
- Conditional **if** b **then** c_1 **else** c_2 : compute WP according to the syntactic rule, use induction on c_1 and c_2 , and reason by cases on whether b holds at m .

This completes the proof sketch for loop-free programs. For loops we rely on the invariant-based characterization described earlier.

7 6. Loops: invariants and variants (termination)

- An *invariant* I for **while** b **do** c satisfies:

$$\{I \wedge b\} c \{I\}.$$

If additionally $(I \wedge \neg b) \Rightarrow Q$ then $\{I\}$ **while** b **do** c $\{Q\}$ holds (partial correctness).

- A *variant* is an expression V mapping states to a well-founded set (typically natural numbers) such that:
 - V is bounded below (e.g., $V(m) \geq 0$ when the precondition holds).
 - Each loop iteration strictly decreases the variant: if $I \wedge b$ holds before the body, then after executing the body V has strictly decreased.

This proves *termination* (total correctness) because an infinite descending sequence cannot exist in well-founded sets like $\mathbb{N}$.

8 7. Functions and recursion

For a (possibly recursive) function f we provide a *contract* $\{\phi\} f \{\psi\}$ and a *variant* V to show termination (for total correctness).

To verify a recursive function:

1. Assume the contract holds for recursive calls (this is akin to induction).
2. Prove that under the contract precondition and the function's body, the postcondition follows (using the assumed contract for recursive calls).
3. For termination, provide a variant V that decreases on each recursive call and is bounded below.

This is standard structural induction over recursion depth.

9 8. Worked examples

9.1 8.1 Addition by repeated increment (from notes)

```
add(a,b) =  
  requires { b \ge 0 }  
  ensures { res = a + b }  
  
  res := a;  
  aux := b;  
  while aux > 0 do  
    res := res + 1;  
    aux := aux - 1;  
  done;  
  res
```

A natural invariant is $I : res + aux = a + b$ combined with $aux \geq 0$. So take:

$$I' \equiv res + aux = a + b \wedge aux \geq 0.$$

We check preservation:

- Assume I' and $aux > 0$ hold. After the body:
 - res becomes $res + 1$,
 - aux becomes $aux - 1$. So $res' + aux' = (res + 1) + (aux - 1) = res + aux = a + b$.
Also $aux' = aux - 1 \geq 0$ because $aux > 0$. Thus I' holds after the body.

On loop exit we have $I' \wedge \neg(aux > 0)$, i.e. $res + aux = a + b$ and $aux \geq 0$ and $aux \leq 0$, so $aux = 0$ and $res = a + b$. Hence the postcondition holds.

Thus we can prove partial correctness. For termination, we can use the variant $V = aux$, which decreases on each iteration and is bounded below by 0.

9.2 8.2 Recursive sum (from notes)

```
sum(n) =  
  requires { n \ge 0 }  
  ensures { res = \sum_{i=1}^n i }  
  if n = 0 then res := 0
```



```

else res := n + sum(n - 1)
endif;
res

```

We prove by induction on n . Contract: $\{n \geq 0\} \text{ sum } \{res = \sum_{i=1}^n i\}$.

- Base: $n = 0$ then $res := 0$ and the postcondition holds.
- Inductive step: assume contract holds for $n - 1$ (recursive call). For $n > 0$, we need $n - 1 \geq 0$ (which holds) and by induction $\text{sum}(n - 1)$ returns $v = \sum_{i=1}^{n-1} i$. Then $res := n + v$ yields $res = n + \sum_{i=1}^{n-1} i = \sum_{i=1}^n i$.

Termination: variant $V = n$ decreases in recursive calls; $n \geq 0$ ensures boundedness.

10 9. Generating verification conditions (VCs)

Most automated verifiers compute $WP(c, Q)$ symbolically (or generate verification conditions) and then hand the resulting logical obligations to an SMT solver. For loops and recursive calls the verifier requires user-supplied invariants and function contracts.

A typical VC generation pipeline:

1. Starting from the desired postcondition Q , compute $WP(c, Q)$ for the whole program.
2. Simplify the predicate.
3. Use rule of consequence to express required preconditions as proof obligations.
4. Send the obligations to an SMT solver (e.g., Z3, CVC4, Alt-Ergo).

11 10. Summary and practical advice

- Hoare logic provides a clean, compositional way to reason about partial correctness.
- For loops and recursion you *must* provide invariants and variants (or contracts and variants) — these cannot in general be discovered automatically.
- Use the assignment axiom and rules of composition, conditional and consequence to structure proofs.
- WP-based VC generation reduces verification to satisfiability/validity problems for SMT solvers.
- Distinguish partial correctness from total correctness: proving termination requires a separate argument (variants/well-foundedness).

12 Appendix A — Full proof of substitution lemma (sketch)

Prove by structural induction on expressions and formulas as outlined in the notes:

- For arithmetic expression constants and variables the lemma is immediate.
- For composite expressions use the inductive hypothesis and the definition of Val .
- For formulas use structural induction over logical connectives and atomic comparisons.

This lemma underpins the soundness of the assignment axiom and the WP-based proof.

13 Closing

This document is intended to be a thorough course-style exposition of Hoare logic suitable for study or conversion to slides/handouts. It follows your request on notation (memory state named $\mathfrak{m}$) and LaTeX delimiters: inline math uses single $\$$ and displayed formulas use $\$$.